

Unit-2

Syntax Analysis: Parsing Natural Language, Tree banks: A Data-Driven Approach to Syntax, Syntax Analysis using Dependency Graph, Syntax Analysis using Phrase Structure Trees, **Parsing Algorithms:** Shift Reduce Parsing, Hyper Graphs and Chart Parsing (CYK Parsing), Models for ambiguity Resolution in Parsing: Probabilistic Context Free Grammar, Generative Models, Discriminative models for Parsing.

Statistical parsing

- Over the last 12 years statistical parsing has succeeded wonderfully!
- NLP researchers have produced a range of (often free, open source) statistical parsers, which can parse *any sentence* and *often get most of it correct*
- These parsers are now a commodity component
- The parsers are still improving year-on-year.

Statistical parsing applications

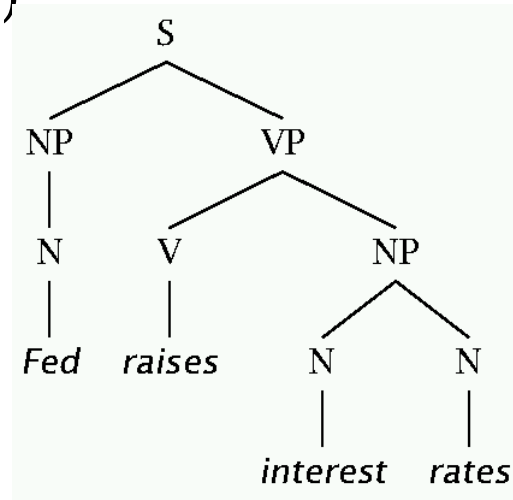
- High precision question answering systems (Pasca and Harabagiu SIGIR 2001)
- Improving biological named entity extraction (Finkel et al. JNLPBA 2004):
- Syntactically based sentence compression (Lin and Wilbur *Inf. Retr.* 2007)
- Extracting people's opinions about products (Bloom et al. NAACL 2007)
- Improved interaction in computer games (Gorniak and Roy, AAAI 2005)
- Helping linguists find data (Resnik et al. BLS 2005)

Two views of linguistic structure:

- Dependency grammar (DG)
- Phrase Structure Grammar (PSG) (Parse Tree)
- PSG breaks sentence into constituents (phrases), which are then broken into smaller constituents. Describe phrase, clause structure Example: NP, PP, VP etc
- DG: syntactic structure consists of lexical items, linked by binary asymmetric relations called dependencies.

1. Tree Bank (phrase structure)

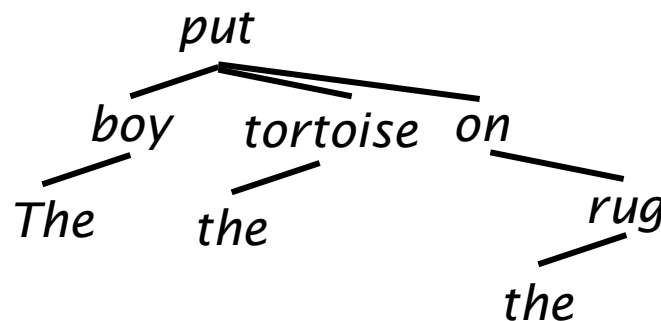
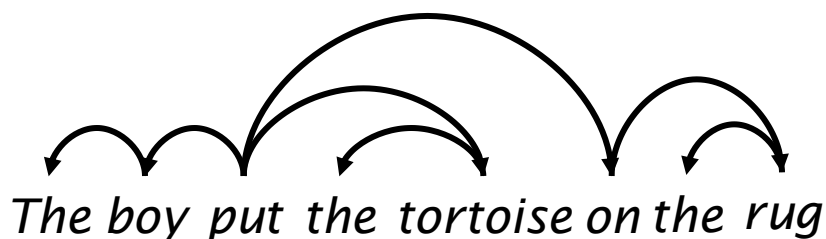
- Phrase structure organizes words into nested *constituents*.
- How do we know what is a constituent? (Not that linguists don't argue about some cases.)
 - Distribution: a constituent behaves as a unit that can appear in different places:
 - *John talked [to the children] [about drugs].*
 - *John talked [about drugs] [to the children].*
 - **John talked drugs to the children about*
 - Substitution/expansion/pro-forms:
 - *I sat [on the box/right on top of the box/there].*
 - Coordination, regular internal structure, no intrusion, fragments, semantics, ...



Two views of linguistic structure:

2. Dependency structure

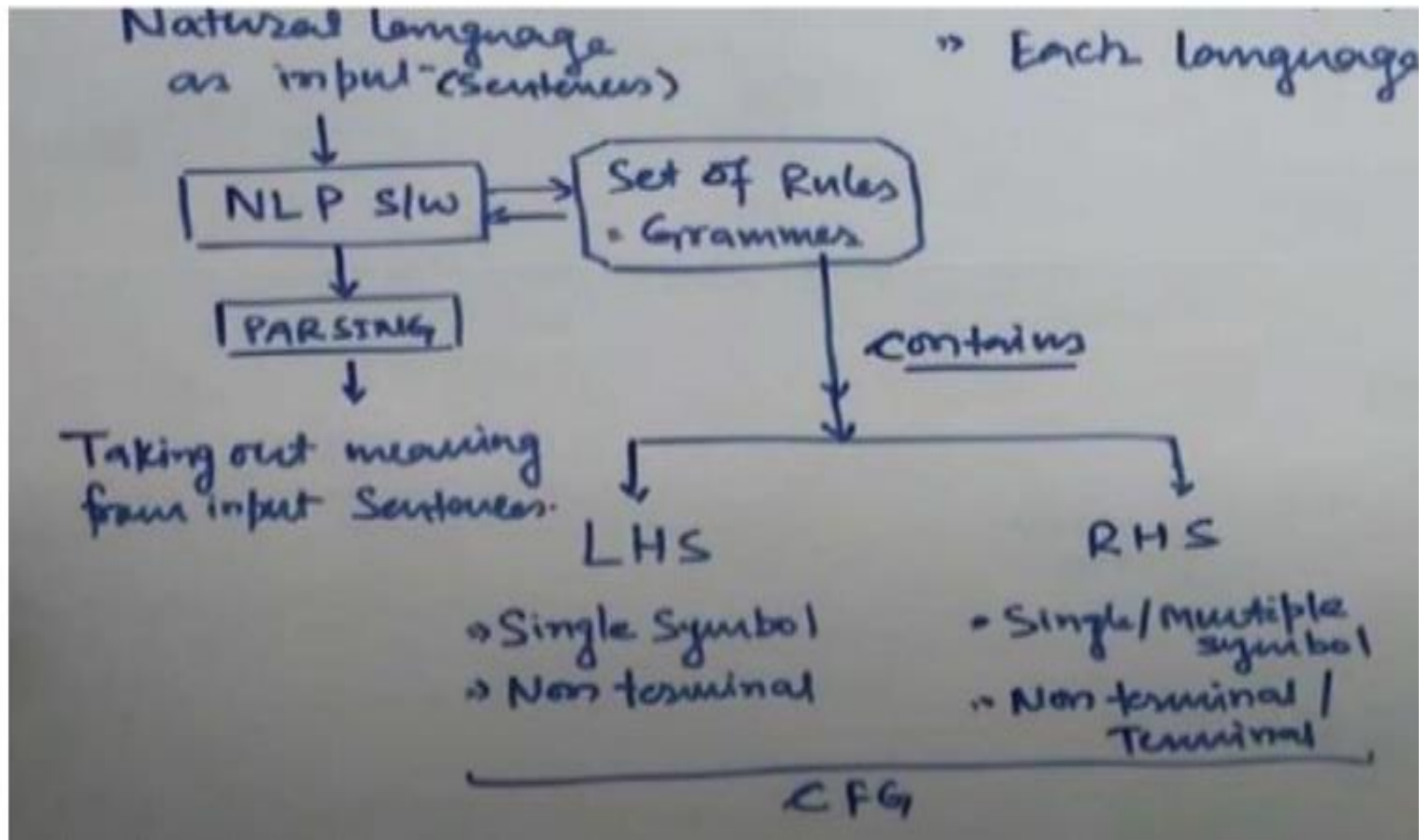
- Dependency structure shows which words depend on (modify or are arguments of) which other words.
- Learning: scoring possible dependency graphs for a given sentence, usually by factoring the graphs into their component arcs
- Parsing: searching for the highest scoring graph for a given sentence



Phrase structure grammars = context-free grammars

- $G = (T, N, S, P)$
 - T is set of terminals
 - N is set of nonterminals
 - For NLP, we usually distinguish out a set $P \subset N$ of preterminals, which always rewrite as terminals
 - S is the start symbol (one of the nonterminals)
 - P is production rules of the form $X \rightarrow \gamma$, where X is a nonterminal and γ is a sequence of terminals and nonterminals (possibly an empty sequence)
- A grammar G generates a language L .

context-free grammars



context-free grammars

- Example: Jogn hit the ball

$S \rightarrow NP VP$

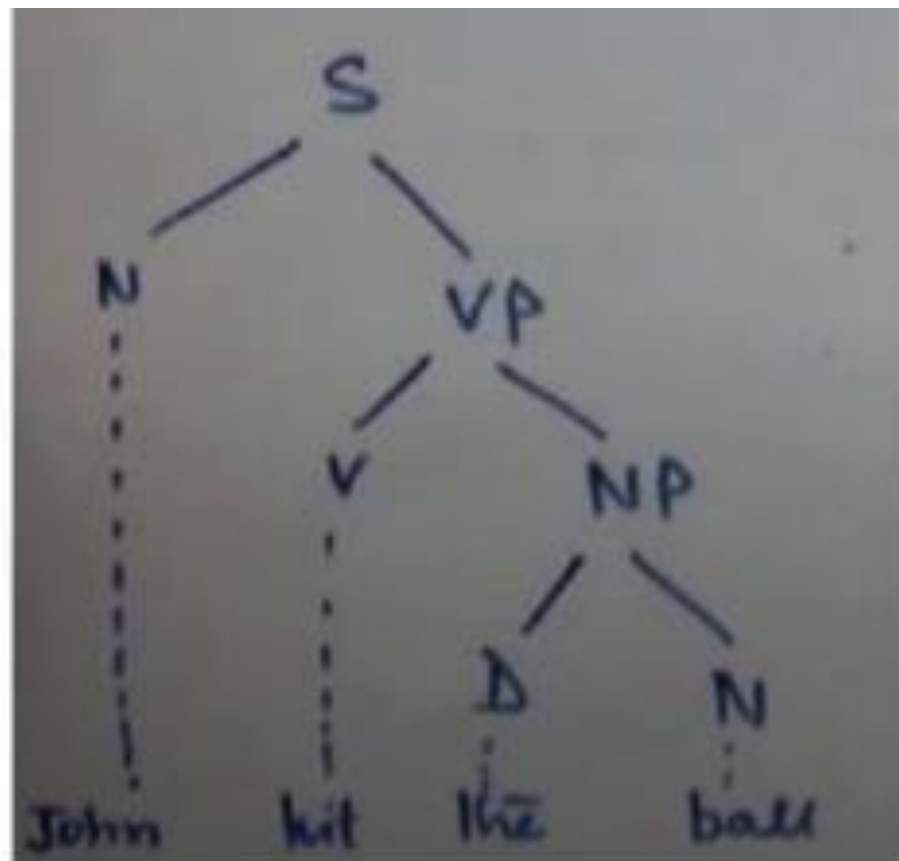
$VP \rightarrow V NP$

$V \rightarrow \text{hit}$

$NP \rightarrow DN$

$D \rightarrow \text{the}$

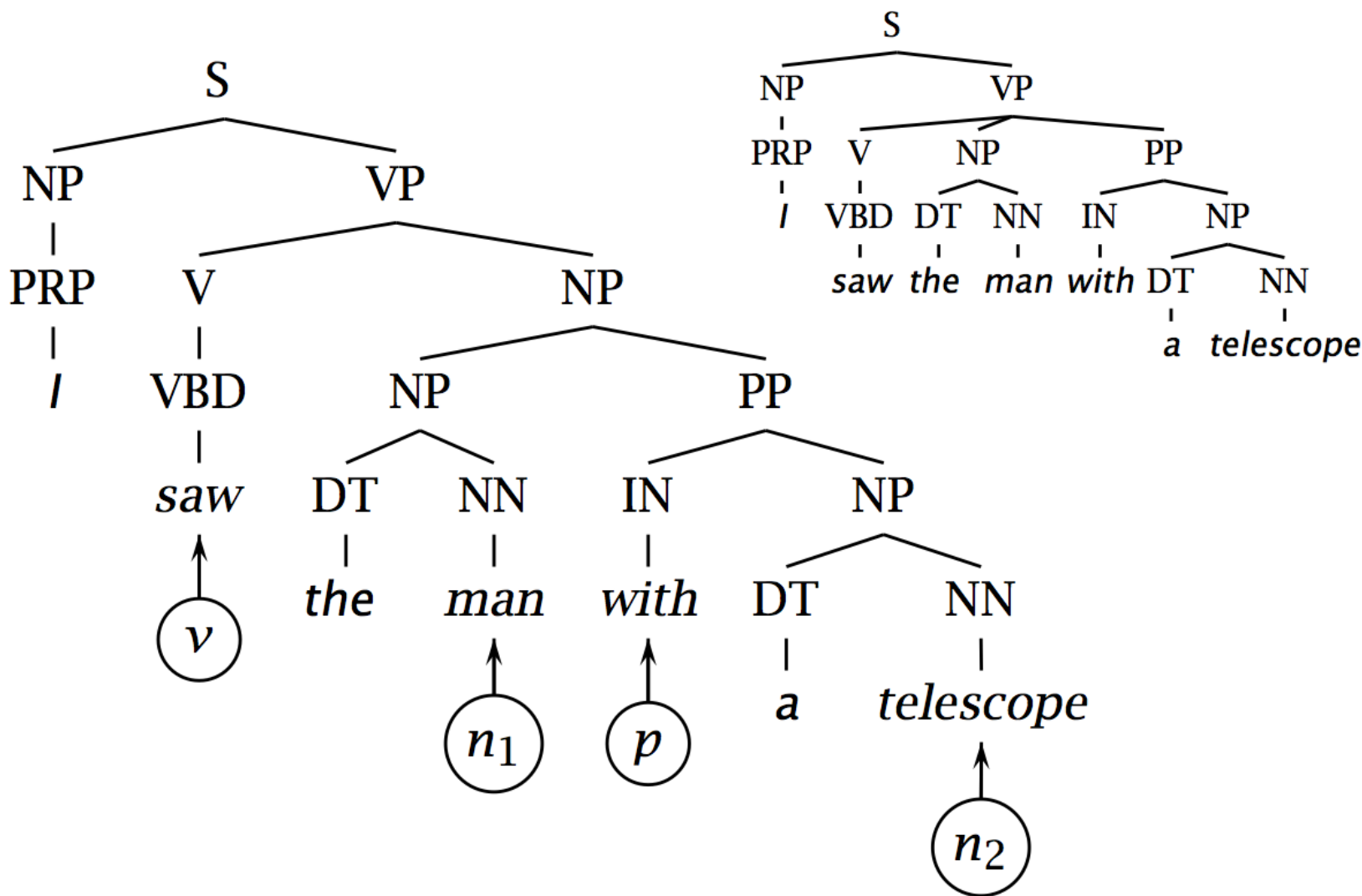
$N \rightarrow \text{John} | \text{ball}$





- S → NP VP
- NP → PRP | DT NN | NP PP
- VP → V NP | V NP PP
- V → VBD
- PP → IN NP
- IN → with
- NN → man | telescope
- DT → the | a
- VBD → saw
- PRP → I

Attachment ambiguities: Two possible PP attachments

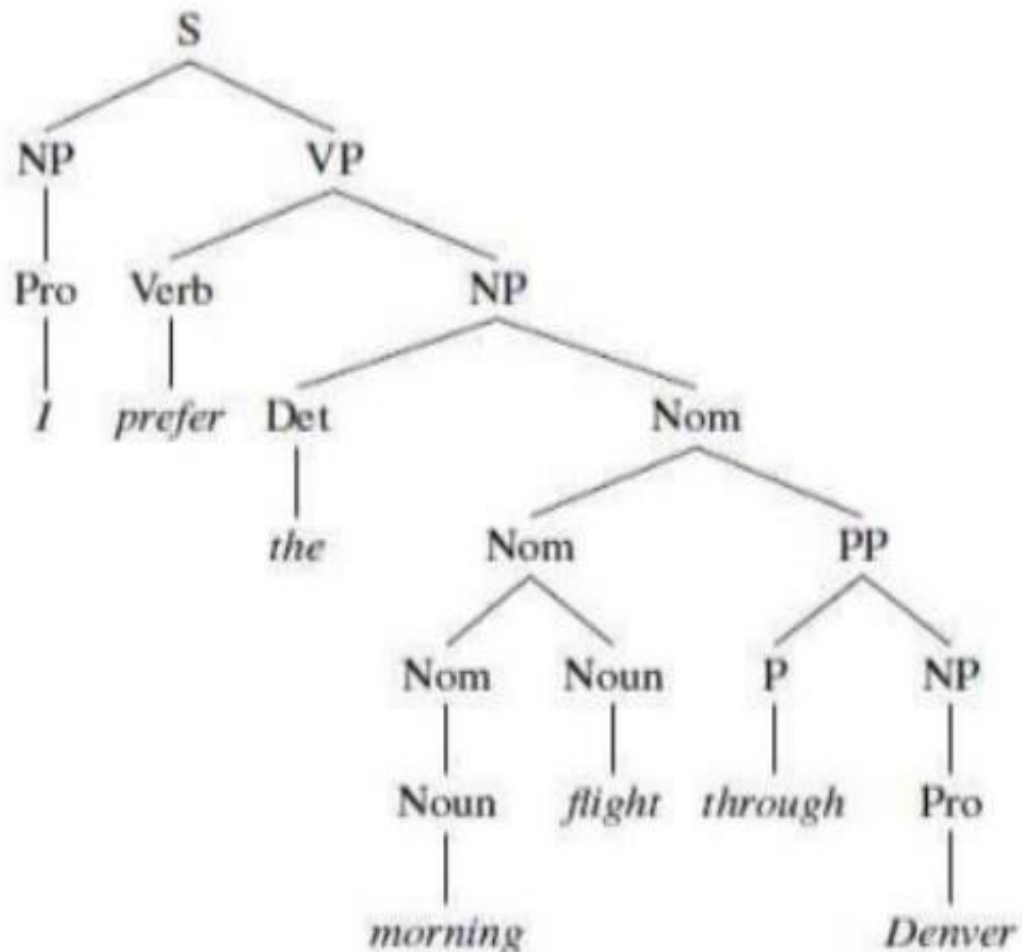
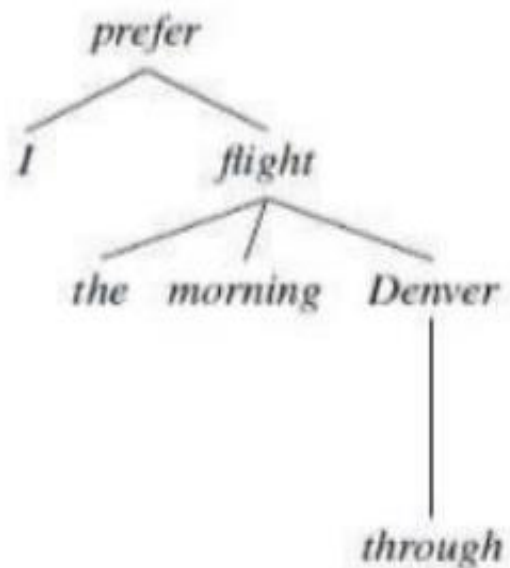


Attachment ambiguities

- The key parsing decision: How do we ‘attach’ various kinds of constituents – PPs, adverbial or participial phrases, coordinations, etc.
- Prepositional phrase attachment:
 - *I saw the man with a telescope*
- What does *with a telescope* modify?
 - The verb *saw*?
 - The noun *man*?
- Is the problem ‘AI complete’? Yes, but ...

What is parsing?

- We want to run a grammar backwards to find possible structures for a sentence
- Parsing can be viewed as a search problem
- Parsing is a hidden data problem
- For the moment, we want to examine *all* structures for a string of words
- We can do this bottom-up or top-down
 - This distinction is independent of depth-first or breadth-first search – we can do either both ways
 - We search by building a *search tree* which is distinct from the *parse tree*



A dependency-style parse alongside the corresponding constituent-based analysis for *I prefer the morning flight through Denver*.

Parsing Algorithms

- Given an input sentence, a parser produces an output analysis of that sentence.
- An important concept for parsing is a derivation.
- For the input string a and b or c , the following sequence of actions separated by symbol represents a sequence of steps called derivation.

Grammar $\rightarrow$

Derivation

$N \rightarrow N \text{ 'and' } N$
 $N \rightarrow N \text{ 'or' } N$
 $N \rightarrow \text{'a' } | \text{'b' } | \text{'c'}$

N
 $\Rightarrow N \text{ 'or' } N$
 $\Rightarrow N \text{ 'or c'}$
 $\Rightarrow N \text{ 'and' } N \text{ 'or c'}$
 $\Rightarrow N \text{ 'and b or c'}$
 $\Rightarrow \text{'a and b or c'}$

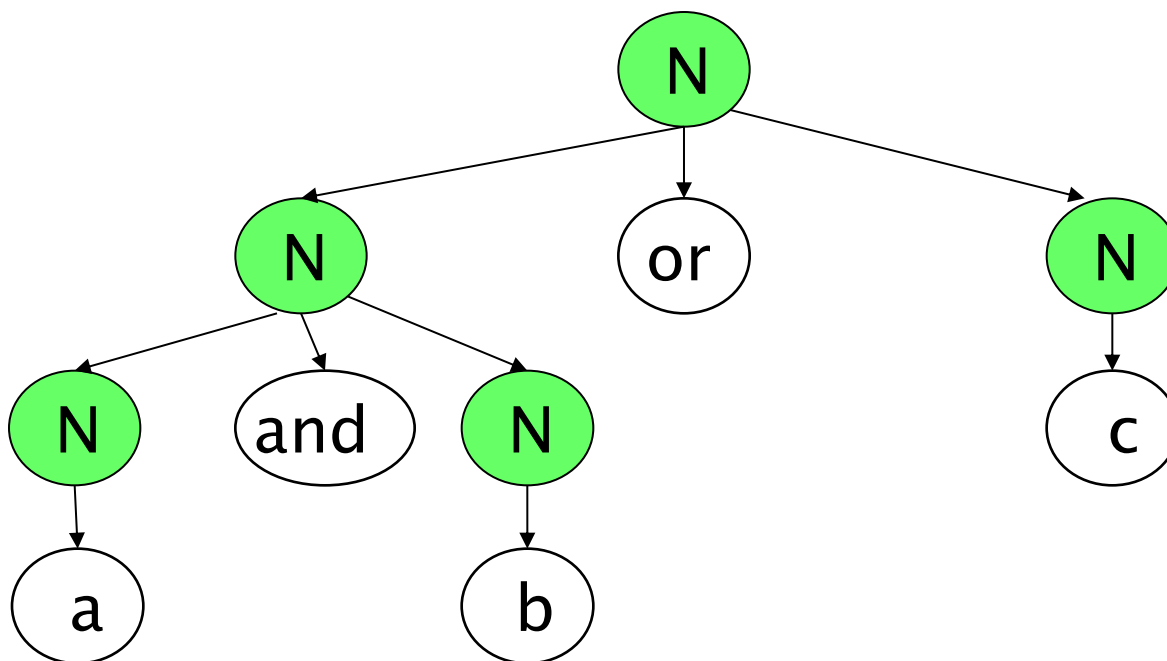
Shift Reduce Parsing

- Start with the sentence to be parsed in an input buffer.
- a "shift" action corresponds to pushing the next input symbol from the buffer onto the stack
- a "reduce" action occurs when we have a rule's RHS on top of the stack.
- To perform the reduction, we pop the rule's RHS off the stack and replace it with the terminal on the LHS of the corresponding rule.
- (either "shift" or "reduce" is possible, choose one arbitrarily.)
- If you end up with only the START symbol on the stack, then success! If you don't, and you cannot and no "shift" or "reduce" actions are possible, backtrack.

$N \rightarrow N \text{ and } N$
 $N \rightarrow N \text{ or } N$
 $N \rightarrow a \mid b \mid c$

Parse Tree	Stack	Input	Action
		a and b or c	Init
a	a	and b or c	Shift a
(N a)	N	and b or c	Reduce $N \rightarrow a$
(N a) and	N and	b or c	Shift and
(N a) and b	N and b	or c	Shift b
(N a) and (N b)	N and N	or c	Reduce $N \rightarrow b$
(N (N a) and (N b))	N	or c	Reduce $N \rightarrow N \text{ and } N$
(N (N a) and (N b)) or	N or	c	Shift or
(N (N a) and (N b)) or c	N or c		Shift c
(N (N a) and (N b)) or (N c)	N or N		Reduce $N \rightarrow c$
(N (N (N a) and (N b)) or (N c))	N		Reduce $N \rightarrow N \text{ or } N$

Parse tree



Dynamic Programming Parsing Methods

- **CKY** (Cocke-Kasami-Younger) algorithm based on bottom-up parsing and requires first normalizing the grammar.
- **Earley parser** is based on top-down parsing and does not require normalizing grammar but is more complex.
- More generally, **chart parsers** retain completed phrases in a chart and can combine top-down and bottom-up search.

Hypergraphs and Chart Parsing (CYK Parsing)

- First grammar must be converted to **Chomsky normal form (CNF)** in which productions must have either exactly 2 non-terminal symbols on the RHS or 1 terminal symbol (lexicon rules).
- Parse bottom-up storing phrases formed from all substrings in a triangular table (chart).

- CFG s in the worst case such a parser might have to resort to backtracking, which means re-parsing the input which leads to a time that is exponential in the grammar size in the worst case.
- ➤ Variants of this algorithm (CYK) are often used in statistical parsers that attempt to search the space of possible parse trees without the limitation of purely left to right parsing.
- ➤ One of the earliest recognition parsing algorithm is CYK (Cocke, Kasami and younger) parsing algorithm and It works only with CNF(Chomsky normal form).

- CFG has lots of problems like ambiguity, backtracking, redundancy
- Need to convert CFG to CNF
- CNF has certain rules
- LHS=RHS
- LHS : non terminal
- RHS contains
 - Single terminal
 - 2 non terminal
 - Combination of terminal and non terminal not possible
 - Single non terminal not possible

- $A \rightarrow a$
- $A \rightarrow B C$
- $NP \rightarrow N PP$
- $NP \rightarrow \text{the } N \dots\dots$ Not possible
- $NP \rightarrow \text{Det Noun PP} \dots\dots$ Not possible

Convert these in to CNF

$NP \rightarrow \text{the } N$

$NP \rightarrow \text{Det Noun PP}$

$-NP \rightarrow \text{Det } N$

$-NP \rightarrow \text{Det Nominal}$

$-\text{Det} \rightarrow \text{the}$

$-\text{Nominal} \rightarrow \text{Noun PP}$

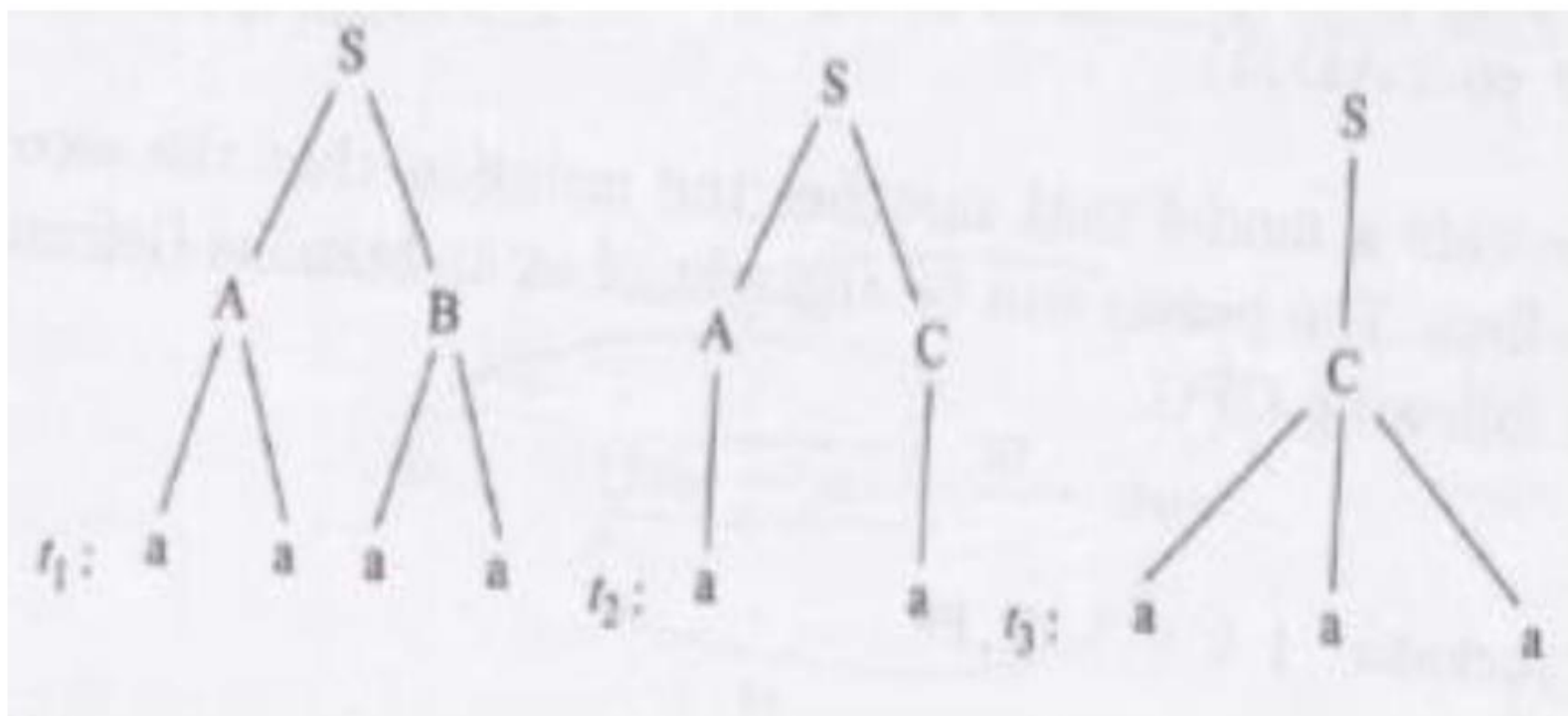
$S \rightarrow NP \ VP \ [0.80]$
 $NP \rightarrow Det \ N \ [0.3]$
 $VP \rightarrow V \ NP \ [0.20]$
 $V \rightarrow includes \ [0.05]$
 $Det \rightarrow the \ [0.4]$
 $Det \rightarrow a \ [0.4]$
 $N \rightarrow meal \ [0.01]$
 $N \rightarrow Flight \ [0.02]$

	1	2	3	4	5
0	Det	NP 0.008			S 0.000 0016
1		N			
2			V		VP 0.000 2
3				Det	NP 0.004
4					N

- The flight includes a meal
- Probability of NP= Det * N * NP =0.4* 0.02* 0.3

Probabilistic context-free grammar

$S \rightarrow A B$ occur 10 times
 $S \rightarrow A C$ occur 20 times
 $S \rightarrow C$ occur 50 times



$$S \rightarrow A B$$

$$S \rightarrow A C$$

$$S \rightarrow C$$

$$A \rightarrow a a$$

$$A \rightarrow a$$

$$B \rightarrow a a$$

$$C \rightarrow a a a$$

$$\frac{10}{10+20+50} = 0.125$$

$$\frac{20}{10+20+50} = 0.25$$

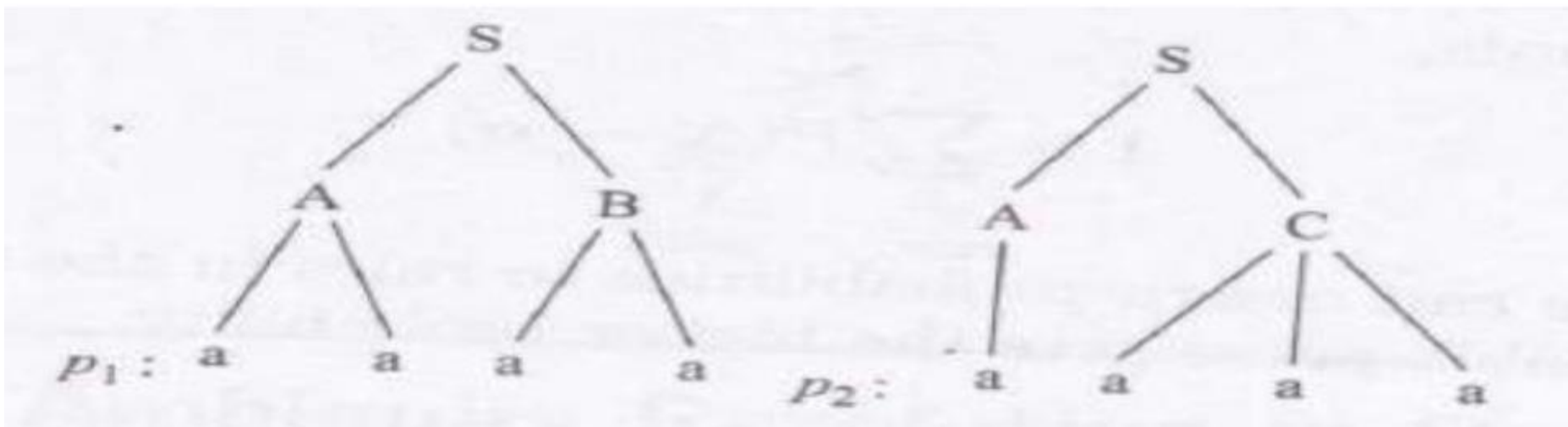
$$\frac{50}{10+20+50} = 0.625$$

$$\frac{10}{10+20} = 0.334$$

$$\frac{20}{10+20} = 0.667$$

$$\frac{20}{20+50} = 0.285$$

$$\frac{50}{20+50} = 0.714$$



- For input a a a there are two parses using the above PCFG: the probability
- $P_1 = 0.125 \cdot 0.334 \cdot 0.285 = 0.01189$
- $p_2 = 0.25 \cdot 0.667 \cdot 0.714 = 0.119$

Generative models

- To find the most plausible parse tree, the parser has to choose between the possible derivations each of which can be represented as a sequence of decisions.
- Let each derivation $D = d_1, d_2, \dots, d_n$, which is the sequence of decisions used to build the parse tree.
- Then for input sentence x , the output parse tree y is defined by the sequence of steps in the derivation.

Generative models

- The conditioning context in the probability $P(d_i | d_1, \dots, d_{i-1})$ is called the history and corresponds to a partially built parse tree (as defined by the derived sequence).
- We make a simplifying assumption that keeps the conditioning context to a finite set by grouping the histories into equivalence classes using a function

$$P(x, y) = P(d_1, \dots, d_n) = \prod_{i=1}^n P(d_i | d_1, \dots, d_{i-1})$$

Discriminative models for Parsing

- Colins created a simple notation and framework that describes various discriminative approaches to learning for parsing.
- This framework is called global linear model.
- Let x be a set of inputs and y be a set of possible outputs that can be a sequence of POS tags or a parse tree or a dependency analysis.
- Each $x \in X$ and $y \in Y$ is mapped to a d -dimensional feature vector $\phi(x, y)$, with each dimension being a real number.
- A weight parameter vector $w \in \mathbb{R}^d$ assigns a weight to each feature in $\phi(x, y)$, representing the importance of that feature

- The value of $\emptyset(x,y).w$ is the score of (x,y) . The higher the score, the more possible it is that y is the output of x .
- The function $GEN(x)$ generates the set of possible outputs y for a given x .
- Having $\emptyset(x,y).w$ and $GEN(x)$ specified, we would like to choose the highest scoring candidate y^* from $GEN(x)$ as the most possible output

$$F(x) = \underset{y \in GEN(x)}{\operatorname{argmax}} p(y \mid x, w)$$

- where $F(x)$ returns the highest scoring output y^* from $GEN(x)$

- A conditional random field (CRF) defines the conditional probability as a linear score for each candidate y and a global normalization term:

$$\log p(y \mid x, \mathbf{w}) = \Phi(x, y) \cdot \mathbf{w} - \log \sum_{y' \in \text{GEN}(x)} \exp(\Phi(x, y') \cdot \mathbf{w})$$

- A simple linear model that ignores the normalization term is:

$$F(x) = \operatorname{argmax}_{y \in \text{GEN}(x)} \Phi(x, y) \cdot \mathbf{w}$$